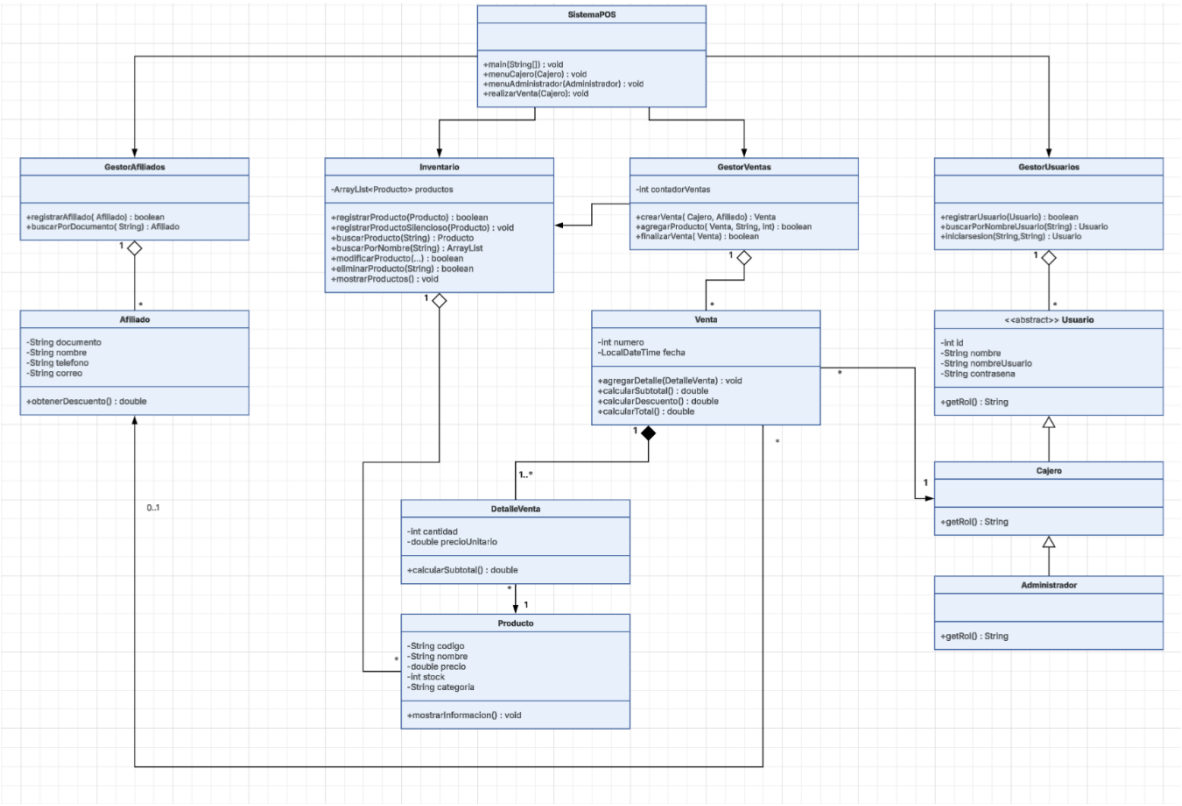


El diagrama muestra cómo está organizado el sistema POS y cómo se relacionan sus diferentes clases.

Para entenderlo, se utilizan varios símbolos: la línea continua indica que dos clases están relacionadas; el triángulo blanco representa la herencia; el rombo blanco indica agregación, es decir, que los objetos pueden existir por separado; y el rombo negro representa composición, donde un objeto depende de otro. También aparecen las multiplicidades, que muestran cuántos objetos pueden participar en cada relación.

En el sistema, SistemaPOS es la clase principal, desde donde se conectan los diferentes gestores que se encargan de manejar los usuarios, productos, afiliados y ventas. Por otro lado, las clases del modelo representan los elementos que hacen parte del funcionamiento del programa.

Una de las relaciones más importantes es la herencia entre Usuario, Cajero y Administrador. También está la composición entre Venta y DetalleVenta, además de las relaciones que tiene una venta con el producto, el cajero y el afiliado.

En general, el diagrama ayuda a entender cómo se distribuyen las clases, qué función cumple cada una y de qué manera se conectan para que todo el sistema funcione correctamente.

Descripción de la aplicación de los pilares de la POO en el proyecto

En el desarrollo del sistema POS se utilizan los cuatro pilares principales de la programación orientada a objetos. Cada uno cumple una función importante en la organización y el funcionamiento del programa:

1. **Abstracción:** Se utiliza en la clase abstracta Usuario, donde se definen los datos y comportamientos que tienen en común los usuarios. Además, se deja el método getRol() para que cada clase que herede de ella lo implemente según corresponda.
2. **Encapsulamiento:** Se aplica al declarar los atributos de las clases como private, evitando que se puedan modificar directamente desde otras clases. Para acceder a ellos o cambiar sus valores se utilizan getters, setters y métodos específicos.
3. **Herencia:** Se encuentra en la relación entre las clases Usuario, Cajero y Administrador. La clase Cajero hereda de Usuario, mientras que Administrador hereda de Cajero. Esto permite aprovechar los atributos y métodos que ya están definidos, sin tener que repetir el código.
4. **Polimorfismo:** Se demuestra con el método getRol(), que tiene una implementación diferente en Cajero y Administrador. Así, cada tipo de usuario puede devolver su propio rol utilizando el mismo método.